Coding Guidelines

My Coding Guidelines

My main principle is: “You will leave; others will come and keep on your work.”

This means I code not for myself, but for future developers. My code must be readable, straightforward, and easy to maintain.

That said, below are all the details I focus on while coding and reviewing PRs, along with their “whys”:

We are Object-Oriented developers.

Always analyse your own code under the shadow of the SOLID principles.

This is my main driving principle when coding. The programming paradigm is Object-Oriented, and there is nothing cleaner or easier to maintain than code that follows the SOLID principles. Always bear in mind:

1. Is the code doing only one thing, or does it have too many responsibilities? Is it “cohesive”? For example, is a message consumer also processing the data and directly calling the producer that publishes it? What if, depending on the data, it shouldn’t be published? What if the process condition changes based on the message type? How long will it take to become an “all-mighty do-it-all class” constantly being modified and growing in size? Follow the Single Responsibility Principle.
2. If some rules change in the method or class, how much code will be affected? Will it be necessary to change existing code, jeopardising all the other code that depends on it? Can I only add a single class with new logic without changing or deleting an existing one? For example: using multiple “ifs” or “switches” to run different, but related logic. If a new related piece of logic comes into place, a new “if” or “case” must be added in the same codebase. Couldn’t I have used any design pattern to prevent this from changing? Follow the Open / Close principle.
3. Are the principles of inheritance and polymorphism being used? Is code reusability and flexibility possible? Can a piece of code continue to work correctly, without changes, if the instance of the dependency it uses is changed, even at runtime? (Liskov Substitution)
4. Is the code depending on “all-mighty classes” with many methods that are not being called? Take the “DateTimeManager” class for example: it has methods to get the current date and time, parse a date to a string, and extract the time from a string. Does the class/method that uses it need all of this, or just the “currentDateTime” feature? Why don’t we make “DateTimeManager” implement something like “CurrentDateTimeIdentifier" and depend only on the second?
5. Is there any break in the layer isolation? Are concrete classes being constantly injected instead of their abstractions? How dependent are the classes on each other, and will it be hard to maintain the Open/Closed principle? (Dependency Inversion)

⚠️ **Important note:**

Often, when using Object-Oriented principles and Design Patterns, someone will come along and say, “This is overengineering,” and cite the famous (but often misused) KISS principle. I prefer this principle without the double “S” (Keep It Simple, not stupid). Remember, it must be simple, not effortless. If someone has not yet taken the time to understand OO, SOLID, and Design Patterns, that’s a gap this person must work to fill. Do not jeopardise future maintenance or the ease of extending and adapting, because now it’s easier to do so by simply adding several “ifs” one after the other (no, a “switch” does not make it better). Go for SOLID every time, and know when exceptions must break that rule.

Keep scopes clear

Do not extend a class only to reuse methods or attributes. Follow the “is a”/“has a” rule (inheritance vs composition).

DRY, Don’t Repeat Yourself

DRY is not only about identical code blocks scattered throughout the codebase; it also means repeating ideas. Placing an identical code block in a method and reusing it in another class doesn’t make it a best practice. I mean, if we were working on a procedural paradigm, yes, but this is OO, so:

1. Check for repetitions that you could easily replace with a design pattern. Check for the most common ones.

2. A code or idea that repeats in the same class can be extracted into a private method.
3. A code or idea that repeats in different classes can be extracted using inheritance or composition (prefer the latter).
4. Test classes are code, too. Avoid repetition there, too. Just because they're tests doesn't mean that they are allowed to be messy! Use inheritance, composition, and `@ParameterizedTests` every time you can.

Avoid creating "utility" classes.

Utility classes are helpful, yes, but they are usually the quick answer to questions we are too busy to look for. The most significant question, which, if answered incorrectly, can jeopardise your architecture, is: "What is the scope of this?" "This" being a class, a method, an attribute, a variable, and so on.

Soon, utility classes become bloated, with classes of varying scopes relying on many static methods. These methods are often duplicated, and often there is a chain of static method calls that makes testing and mocking difficult. We risk creating a refactoring nightmare.

Use public static methods judiciously (usually never)

This guideline is strongly related to the "utility" classes one. My rule of thumb is "avoid public static methods". The only reason to have it is if you are creating a library without using a dependency injection framework, such as Spring. For example, a factory method is sometimes used to avoid calling the constructor.

Using a public static method "just because" may cause problems in the future, it creates a strong dependency between classes, it prevents, or makes it harder to use polimorphism and Liskov Substitution and even though Mockito can help you test it using `Mockito.mockStatic(Schema.class)`, it will initialise the class being mocked, and possibly running static code that wasn't supposed to run in a test scope.

The solution: Create a simple class; leave instantiation and lifecycle handling to the class/system that uses it. It can be a simple `@Bean` in a Spring-managed system, or part of some Abstract Factory pattern.

Code style and standards

First of all, it is essential to highlight that nothing is set in stone, and in some cases, rules need to be overthrown. But do it judiciously and do not transform the exception into a rule.

Identify the current code style and either replicate or adapt it as a whole.

This is especially true if you start working with a team that has been working with the codebase for some time.

For people who come long after the original developers are gone, standardisation, patterns, and code styles that repeat throughout the code are easy to identify, understand, and therefore maintain.

Try to understand the "whys" of how something has been coded and, assuming it is clean code, keep the same standards as the rest of the code. If the team agrees that some standards must change, regardless of the reason, start changing them until the entire code looks the same. If there is any substantial disagreement with a particular style, communicate. Good arguments regarding code standards enrich the whole team.

Pay attention to modifiers and other keywords.

Modifiers (public, private, "default", final, static, etc.) send a message to readers and, especially, maintainers. With them, you express the need for encapsulation, immutability, and more. Sending such a message helps others understand the architectural intent behind your code, so use it correctly.

NOTE: Modifiers are not limited to business logic code; test code must also be considered.

1. Do I want all other classes to be aware of this attribute? If so, use `public`.
2. Do I want other classes in the same package to be aware of this attribute? If not, use `private`.
3. Will this attribute be changed? If not, use `final`.
4. Do I need a different instance of this attribute for every instance of this class? If not, use `static`.

Use the `final` keyword for variables and parameters that are not meant to be reassigned.

As for code standards, the `final` keyword is mandatory for method parameters and assignments **that are not reassigned in the code that uses it**.

What are the advantages of using `final` if it makes no difference regarding performance?

1. It improves readability, especially in professional IDEs that highlight keywords, visually distinguishing assignments from other operations.
2. It aids code maintenance, since it is clear that the value of a variable or parameter doesn't change in some shady part of the code that has slipped my eye.
3. Allowing over-reassignment of parameters and variables is a bad practice that harms readability and maintenance.
4. Sends the reader (usually the maintainer) a message: "I don't want this parameter or variable to be reassigned".

Use the `var` keyword unless declaring the variable's type improves readability.

I understand why some developers do not like using `var`; I do not entirely agree with that, so below, I present my thoughts on the matter:

1. Not using `var` increases readability by making it clear at a glance what type of variable is being assigned. Very useful when the value is assigned via a complex chained method call.
2. Simply assigning values to variables or giving them meaningful names usually makes the data type obvious; therefore, type identification becomes redundant. In those cases, use `var`.
3. Some data types, especially when using Generics, can become very verbose, leading to variable assignments that exceed the maximum line length and require line breaks, making the code difficult to read. Use `var`.

Blank lines

That's totally my opinion, and my opinion only. Ignore at will, but I will change it if I have to maintain the code block where the extra line is located:

1. Although it poses no practical problem, GitHub "complains", showing a "prohibited" emoji at the end of the file if a blank line is missing. It is just my OCD talking, but I don't like that forbidden sign in my code when I review it on GitHub.
2. Regarding readability, I don't see a good reason for extra blank lines between methods (e.g., 2 or 3 blank lines separating them), since the IDE can add a visual line separator, making the separation more straightforward to identify.
3. I always put one blank line right after the class declaration and one before the class closing bracket.
4. I never put a blank line either right after the method signature or before the method's closing bracket.

Use Text Blocks for readability.

For large strings with line breaks (`\n`), use text blocks (`"""`) instead of concatenating them with the plus sign (`+`) or using StringBuilder/StringBuffer.

For large strings without line breaks, use `\` after the end of each line. It will treat the whole text as a single, unbroken line:

```
final var someText = """
   Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod \
   tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, \
   quis nostrud...""";
```

Avoid "Magic Numbers"

Numeric literals that are not named by a constant declaration can lead to unclear code and errors if a magic number is changed in one location but left unchanged in another.

Keep variable declarations as close as possible to their use.

- Improved Readability: Variable declaration immediately followed by its usage - clear intent
- Reduced Cognitive Load: No need to scroll up to understand what a variable is
- Better Code Flow: Logical grouping of related setup (mock → when → use)
- Easier Debugging: When a test fails, the related mock and its setup are together
- Follows Best Practices: Variables are scoped as tightly as possible.

Documenting

First of all, I must cite the Agile Manifesto:

> Working software over comprehensive documentation

and

> The most efficient and effective method of conveying information to and within a development team is face-to-face conversation.

That said... Although I agree with the Agile Manifesto and disagree with mandatory documentation, I fully support it for certain types of codebases, such as reusable services and libraries among teams, or complex flows.

Javadoc is for the future.

As for our code standards, Javadoc is mandatory for classes, interfaces, records, enums, and public and protected methods.

These are the guidelines regarding Javadoc that I follow every time I add code:

1. Check for missing Javadoc. They are mandatory in classes (including interfaces, records, enums, ...) and public and protected methods. Exception: Test classes and their methods do not need Javadoc (the idea is that tests are part of the documentation, at least in theory).
2. Review the Javadoc and keep it up to date. Remember, this text is not for you, but for maintainers who will come in the future. There is no problem with using AI to generate Javadoc or copying and pasting Javadoc from similar classes or methods; the problem is not reviewing it afterwards.
3. New generics in class definitions and new method parameters require Javadoc updates. Pay attention when updating existing signatures.

Keep the README updated.

Sometimes a simple “what does this project do?” is enough. But if you are going to maintain someone else’s code, it would be nice to have some tips on how to run, add or fix the code without breaking the original architectural idea. It can be a set of basic directives or a simple diagram depicting the architecture.

Those things go well in the README, and if anything more specific is needed, or if the company demands centralised documentation (in a Confluence-like tool, for example), add links!

Java issues

Do not use `Double`/`double` for mathematical calculations; use `BigDecimal`.

Be aware that Java (among other languages) have precision issues when working with doubles in mathematical operations. Depending on the precision level, use BigDecimal instead. For more, check the following article:

- Why Double Loses Precision and How to Avoid It in Java

Streams' iterations are slower than old-fashioned loops.

-> Check “Give `Streams` a try”.

Going functional

Give `Optional` a try

Note: Use it to improve readability and maintainability, but do so with caution.

1. Use `Optional.ofNullable( . . . ).orElse( . . . )` instead of `if ( . . . != null) { . . . }`.
2. Instead of `obj.getX().getY().getZ()`, which can raise an unwanted `NullPointerException`, use `Optional.ofNullable(obj).map(Obj::getX).map(X::getY).map(Y::getZ).orElse( . . . )`.
3. Do not over-nest `Optional`, since it reduces readability, for example:
    a. `Optional.ofNullable( . . . ).orElseGet(( ) -> Optional.ofNullable( . . . ).orElseGet(( ) Optional.ofNullable( . . . ).orElse( . . . )`.

Give `Streams` a try

Note: Use it to improve readability and maintainability, but do so judiciously.

Streams are constantly evolving to become the first choice for iteration, but in Java 25, they are still slower than regular loops for small sets, though they enhance code readability. Put both on a balance and see whether it is really disadvantageous to use streams; if not, favour them.

Refusing to use streams because of a third-party benchmark that said it is slower than a normal for loop, or because you are already used to writing for/while loops, and think that Streams are too complicated, will only get you outdated as fast as the language evolves.

Testing

Go for 100% coverage.

Not 99%, 100%! Why? As simple as “If some code can’t be covered, it is usually poorly designed.” Want other reasons?

1. Every time you leave code untested, you set an example for others, bringing the system closer to SonarQube’s acceptable threshold. Someday, someone (not you, but people who will maintain today's code) will need to leave code untested (for whatever reason), and they won’t be able to do so, risking production deadlines or disabling SonarQube and totally jeopardising code quality.
2. Every time you leave code untested, you lose an excellent opportunity to analyse your own code and learn about module dependencies and coverage rules, how libraries like Lombok create code behind the scenes, how to increase decoupling and cohesion, apply design patterns and SOLID principles, and a lot of other things that would make your code easier to maintain and you a better developer.

But… And this is a big “but”… If you really need to leave something untested, explain the reason and be honest about it. Leave a comment on your PR or even the code. Perhaps someone can help you understand how to improve it.

Do not use the tested logic to assert results; use mocks.

Usually, the logic to create the expected result object is the same as that used by the code being tested, usually in the form of “builders” (that’s just an example). Builders are the main building blocks of the data our business logic receives, processes, and provides. Using them to create the object we are going to compare in the assertion may replicate an error if the builder assembles the object incorrectly and provide a “false positive” in the tests. For very complex objects, it also becomes hard to understand what is really being asserted there.

Use Mockito's “mock” and “when” to explicitly state what you expect from the object being tested. To assert a whole object against the mocked object, use AssertJ’s `usingRecursiveComparison`.

Do not make a method `public` to ease testing.

The scope and visibility of a method are determined by its objective within the class in which it belongs. If you need to make a method public because you cannot test all scenarios and/or increase coverage, it means your code is not testable

and therefore has design flaws. Try favouring composition. Extract this method into another class and use polymorphism with the Strategy design pattern.

Use Mockito judiciously to enhance your test quality.

Beware of `@Mock` and `@InjectMocks` :

Mockito is the market leader in mocking classes and achieving real Unit Testing, and its annotations help us eliminate a lot of boilerplate code. However, when you give a framework the power to control how it instantiates and injects objects, you may also lose control of how things work in the background.

For example, Spring has long advocated constructor-based dependency injection over `@Autowire`. why? Mainly because it makes dependencies **explicit**, promoting a more robust, clean, and testable design. When using `@InjectMocks`, for example, you are moving in the opposite direction.

A practical and simple example:

Class `A` has a constructor that depends on `B` and `C`. `B` and `C` are annotated with `@Mock`, and `A` is annotated with `@InjectMocks`. A simple refactoring removes the `C` dependency. The compiler will not complain, but the test will fail.

Yes, the test will fail; there is nothing to worry about. However, the required change to the test **is not explicit**. So you lose the compiler's help, and the code fails to communicate .

About `any()`, `any(class)`, `anyString()`, and so on:

- When mocking behaviours (using Mockito's `when`), favour `any(SomeClass.class)` or its wrappers (e.g. `anyString()` instead of simply `any()`. When working with polymorphism, disregarding the parameter's value may be acceptable, but it may lead to false positives.
- For the same reason as before, when verifying that some method from some class was called, favour `any(SomeClass.class)` or its wrappers (e.g. `anyString()`) instead of simply `any()`.
- However, when verifying that the method was not called, use only `any()` to ensure it hasn't been called at all with any parameter value, disregarding its type.

To increase readability:

- Use `never()` instead of `times(0)` to check that a single method has never been called.
- Use `verifyNoInteractions` to check that the whole object had no part in the logic being tested.

Avoid conditionals in tests.

Our code must be deterministic; the same input must produce the same output every time. Putting an "if … else" during a test can mean two things: 1) You don't know what result may come from some computational logic, or 2) your code is so hard to test that you can't really control the outcome of a method.

If you cannot replace your "if result is this, assert this way, if result is that, assert that instead" with a `@ParameterizedTest`, then your design may have a flaw and needs to be reviewed.

Committing your work

Try to keep commits clear.

1. Always update the Jira ticket. It helps identify what work the change was related to. When you are long gone, it can help others identify the broader scope of the problem you were solving and find colleagues who are still around and understand the code you were maintaining.
2. Try to create small commits grouped by scopes.
3. Always put a meaningful commit message. It doesn't have to be a book; it should make it easier to understand, identify, and track changes.
4. Unless you have a GitHub commit hook that runs best-practice checks and SonarQube reviews when you try to commit your changes, don't do it in the Terminal; use the IDE. It will warn you if anything out of the ordinary is found and prevent you from committing it without noticing.

Take advantage of the IDE help (Warnings and Commit Inspections)

1. The IDE alerts on bad practices and potential future problems. Do not let these warnings go unattended! Address all warnings the IDE displays. It may be a silly one, but most of them will prevent many issues in the future.
2. If there is a warning that you are sure must be disregarded, use the appropriate "@SuppressWarning" to prevent the IDE from showing it again. With time, the list of unattended warnings becomes enormous, and you can no longer separate the warnings that you should pay attention to from those you should disregard.
3. Use the SonarQube plugin; bind it to the project's server rules. It can anticipate many issues and "bad practices" before you commit and push to the repository.
4. Unless you have a GitHub commit hook that runs best-practice checks and SonarQube reviews when you try to commit your changes, don't do it in the Terminal; use the IDE. It will warn you if anything out of the ordinary is found and prevent you from committing it without noticing.

Aiding Tools

Maximise IntelliJ IDEA's help

- Organise Imports (Ctrl + Opt + O)
- Format file (Cmd + Opt + L)
- Avoid "*" on imports (set Settings > Editor > Code Style > Java > "Class count to use import with *" *and "Names count to use static import with **" to 999)
- Avoid magic numbers: check Settings > Editor > Inspections > java > Abstraction issues > Magic numbers
- Suggest using the `final` keyword: check Settings > Editor > Inspections > java > ...
  - ... Class structure > Non-final field in 'enum' > ...
  - ... Code style issues > ...
    - ... Field may be 'final'
    - ... Local variable or parameter can be 'final'
- Pay attention to the IDE's warnings.
- Use the SonarQube plugin and set it to the project's SonarQube server.

AI is meant to help...

... But not to do your job. Use it! Vibe code with it! Be spec-driven! Be a "Harness Engineer"! But review the result thoroughly. A simple sign that someone used AI without reviewing is the total disarray in the code, whether in the standards used or in the lack of reuse.

This document is an eternal Work in Progress...

TBD.